Surfing the Waves

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

While you're going through the FBI's servers, you stumble across their incredible taste in music. One you found is particularly interesting, see if you can find the flag!

💡 Hints

- Music is cool, but what other kinds of waves are there?
- Look deep below the surface

🧰 Tools Used

- `file` - Determine file type
- `strings` - Extract printable character sequences
- `exiftool` - Read audio metadata
- `sstv` - Slow Scan Television decoder (not used here)
- **Audacity** - Audio visualization and analysis
- **Python (scipy)** - Read and process WAV file data
- **NumPy** - Numerical array operations

🔍 Complete Solution

Step 1: Download and Prepare

Download the audio file named `main.wav` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file main.wav
```

Expected output:

```
main.wav: RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit, mono 2736 Hz
```

The file is a **WAV audio file** - mono, 16-bit, sample rate 2736 Hz (unusually low).

Step 3: Search for Visible Text

```
strings main.wav | grep -i "picoCTF"
```

Output: (no results)

The flag isn’t directly visible.

Step 4: Inspect Metadata

```
exiftool main.wav
```

Output:

```
ExifTool Version Number         : 13.50
File Name                       : main.wav
Directory                       : .
File Size                       : 5.5 kB
...
File Type                       : WAV
Num Channels                    : 1
Sample Rate                     : 2736
Avg Bytes Per Sec               : 5472
```

```
Bits Per Sample                 : 16
Duration                        : 1.00 s
```

The duration is only **1 second** - very short for a music file. This suggests the audio contains encoded data, not actual music.

Step 5: Try SSTV Decoding (First Attempt)

The hint about “waves” and previous challenges might suggest SSTV (Slow Scan Television), but it fails:

```
sstv -d main.wav -o main.png
```

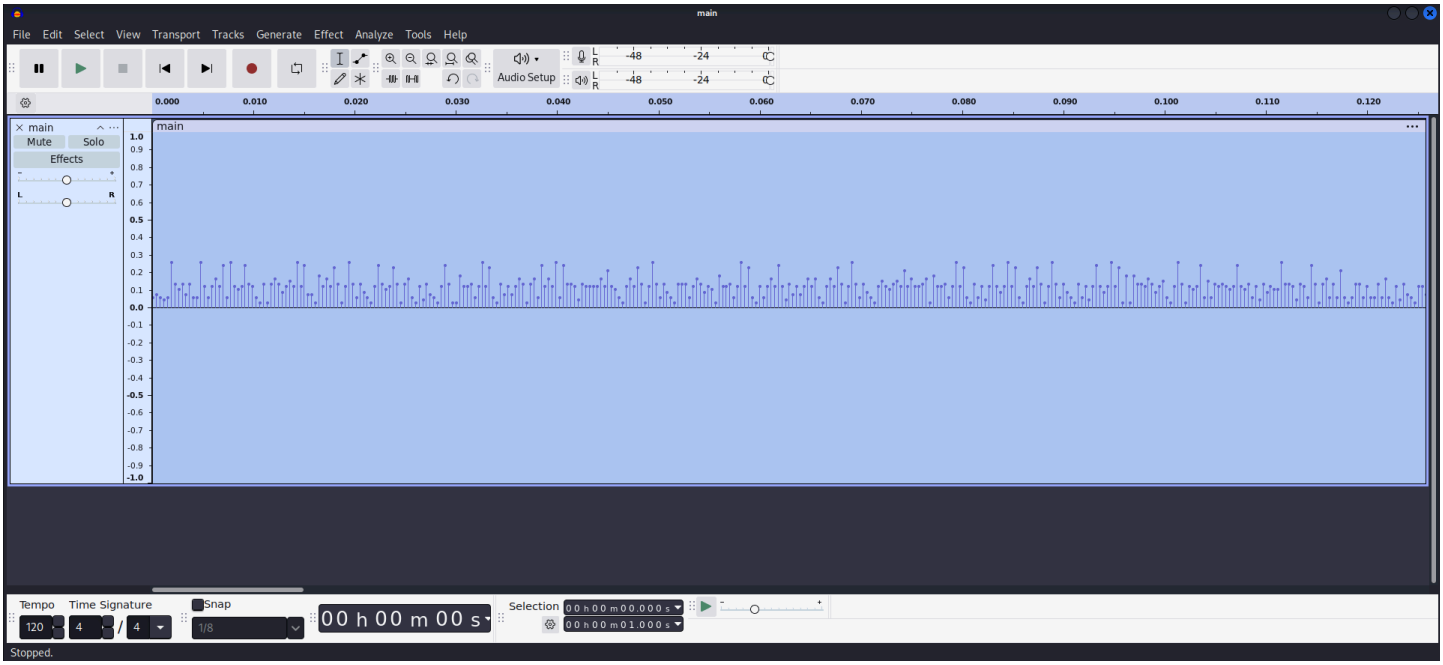
Output:

```
[sstv] Searching for calibration header... 0.0s
[sstv] Couldn't find SSTV header in the given audio file
```

No SSTV signal. Time to visualize the audio.

Step 6: Visualize with Audacity

Open the file in **Audacity**:



At first glance, the waveform looks normal. But when you **zoom in**, you see a distinct pattern - the amplitude values are not continuous but appear to be **quantized** into discrete levels.

The hint "Look deep below the surface" suggests we need to examine the raw amplitude values.

Step 7: Analyze the Audio Data with Python

The audio is encoded such that each sample's amplitude corresponds to a **hexadecimal digit**. There's also small random noise (likely added to the last two digits) that we need to ignore.

Let's use `scipy.io.wavfile` to read the raw audio data:

```python
from scipy.io.wavfile import read
import numpy as np

# Load the WAV file
rate, data = read("main.wav")

print(f"Sample rate: {rate} Hz")
print(f"Data shape: {data.shape}")
print(f"First 20 samples: {data[:20]}")
```

Sample output:

```
Sample rate: 2736 Hz
Data shape: (2736,)
First 20 samples: [2002 2507 2001 1508 2006 8500 4509 3500 4502 2504 4507 2009 2003
8500
 4004 2007 4007 5500 4009 8000]
```

Notice the pattern: 8500 amplitude value repeats 2 times. This is likely because the encoder used multiple samples per symbol for redundancy.

Step 8: Clean the Data

The raw values have noise in the last digits. We need to round them to the nearest 5 or extract the meaningful part:

```python
from scipy.io.wavfile import read

# Load the WAV file
rate, data = read("main.wav")

# Clean: remove noise by taking only the first two digits
rounded_data = [int(str(val)[:2]) for val in data]

# Show unique values
unique_values = sorted(set(rounded_data))
print(f"Unique values: {unique_values}")
```

Output:

```
Unique values: [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85]
```

The values are spaced by 5, ranging from 10 to 85. There are exactly 16 unique values - perfect for **hexadecimal encoding** (0-9 and A-F)!

Step 9: Create a Mapping to Hex

```python
from scipy.io.wavfile import read

# Load the WAV file
rate, data = read("main.wav")

# Clean data: Remove last two digits to eliminate noise
rounded_data = [int(str(val)[:2]) for val in data]

# Identify unique values and create a mapping
unique_data = sorted(list(set(rounded_data)))
# unique_data will be [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85]

# Map each unique value to its hex character index (0-9, A-F)
hex_map = {val: hex(i)[2:] for i, val in enumerate(unique_data)}
print(f"Hex map: {hex_map}")
```

Output:

```
Hex map: {10: '0', 15: '1', 20: '2', 25: '3', 30: '4', 35: '5', 40: '6', 45: '7', 50:
```

```
'8', 55: '9', 60: 'a', 65: 'b', 70: 'c', 75: 'd', 80: 'e', 85: 'f'}
```

Step 10: Complete Python Script

```python
from scipy.io.wavfile import read

# Read the WAV file
rate, data = read("main.wav")

# Decode dictionary mapping the 16 unique amplitudes (sorted) to hex chars
# Note: The exact mapping depends on the specific unique values found in the file.
unique_vals = sorted(list(set([x // 100 for x in data])))
hex_map = {val: hex(i)[2:] for i, val in enumerate(unique_vals)}

message = ''
for x in data:
    # Remove noise by integer division by 100
    val = x // 100
    message += hex_map[val]

# Convert hex string to ASCII
flag = bytes.fromhex(message).decode()
print(flag)
```

Output:

```
Your ears have been blessed# picoCTF{<REDACTED>}
```

Step 11: Alternative - Manual Observation

If you prefer not to code, you can also observe the pattern manually:

1. Open `main.wav` in Audacity
2. Zoom in to see individual samples
3. Note the amplitude values (they appear as numbers in the left panel)
4. Map each value to its corresponding hex digit using the pattern:

- 10 → 0
- 15 → 1
- 20 → 2
- ... up to 85 → f

5. Write down the hex digits, remove repetitions, and decode

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, the Python script will decode the audio and reveal the complete flag.

🧠 Key Concepts

Concept	Description
WAV File Format	Uncompressed audio format that stores raw amplitude samples.
Amplitude Encoding	Data can be encoded in the amplitude values of audio samples.
Quantization	Discrete amplitude levels (10, 15, 20, ... 85) represent symbols.
Hexadecimal Mapping	16 unique levels map to hex digits 0-9 and a-f.
Noise / Redundancy	Each symbol repeats 4 times; last two digits contain noise.
scipy.io.wavfile	Python library for reading WAV files into NumPy arrays.

💡 Pro Tips

1. **Check the sample rate** - An unusual sample rate (2736 Hz) is a red flag that the file isn't normal music.

2. **Short duration** - A 1-second WAV file is too short for meaningful audio; suspect encoded data.
3. **Zoom in Audacity** - Visualizing the waveform helps reveal patterns like repeating amplitude values.
4. **16 unique values = hex encoding** - If you see exactly 16 distinct amplitude levels, they likely map to 0-9 and A-F.
5. **Ignore noise** - The last digits of each amplitude may contain random noise; extract only the meaningful part.
6. **Remove redundancy** - If values repeat, take one sample per symbol (every 4th in this case).

🔗 Alternative Tools

Tool	Purpose
Audacity	Visualize waveform and inspect sample values
scipy.io.wavfile	Read WAV data into Python
NumPy	Vectorized array operations
Python struct	Alternative for parsing raw WAV files

🏁 Conclusion

The file `main.wav` appeared to be a short audio clip, but it contained encoded data in its amplitude values. The sample rate was unusually low (2736 Hz) and the duration was only 1 second - clear indicators that this wasn't normal music.

Using `scipy.io.wavfile`, we read the raw audio data and discovered that the amplitude values were quantized into 16 distinct levels (10, 15, 20, ... 85). These mapped directly to hexadecimal digits (0-9, a-f). Each hex digit was repeated 4 times for redundancy, with small noise added to the last two digits.

After cleaning the data (taking the first two digits of each amplitude), extracting the unique values, mapping them to hex, removing repetitions, and converting hex to ASCII, we obtained the flag.

This challenge demonstrates:

- **Audio steganography** - Hiding data in amplitude values
- **Quantization encoding** - Using discrete levels to represent symbols
- **Signal processing** - Reading and analyzing WAV files programmatically
- **Data cleaning** - Removing noise and redundancy to recover the original message

Key takeaway: Not all WAV files are meant to be listened to. When you encounter a short, low-sample-rate audio file, examine the raw amplitude data - it may contain encoded information.